

Informática II

Programación Orientada a Objetos: Constructores y Destruyores

Profesor Dr. Paul Bustamante

Índice

1. Repaso POO
2. Constructor
3. Destructor
4. Puntero implícito *this*
5. Miembros *static* de una clase
6. Arrays de objetos
7. Funciones *amigas*

Repaso - Organización de ficheros

Programa.cpp

- Contiene el programa principal con acciones sobre las clases

Nombre clase.h

- Contiene la definición de la clase

Nombre clase.cpp

- Contienen la definición de las funciones y operadores de la clase

Repaso - Concepto de “clase” en C++

Una **Clase** es un ***tipo de datos*** definido por el usuario.

Básicamente, es una agrupación de datos (**variables**) y de funciones (**métodos**) que operan sobre esos datos.

Por lo general, las definiciones de las clases se hacen en ficheros de cabecera (*.h), los cuales suelen llevar el nombre de la clase, en el caso de que sólo haya una en él.

- Los **miembros** de una clase pueden ser **variables** de cualquier tipo y **funciones**.
 - a las variables se las denomina *datos miembro*
 - a las funciones, *funciones miembro* de la clase.
- **Datos miembro de una clase (fichero *.h):**
 - Para declarar un dato miembro se procede de la misma forma que para declarar cualquier variable.

```
class Complejo {  
    public:                                //especificador de acceso  
        float real;                       //Declaración correcta  
        float imaginario = 0;            //Inicializar  
};
```

Para acceder a un dato miembro (siempre y cuando sea declarado como **público**) de una clase desde un objeto, se utilizará el operador Punto (.).

Ejemplo: *c1.real = 5.4;*

Repaso - Funciones miembro de una clase

Las funciones miembro de una clase **definen las operaciones que se pueden realizar con sus datos miembro.**

```
class Complejo{  
    private:  
        float real, imaginario;  
    public:                                //funciones miembro públicas  
        void Asignar (float x, float y);  
};
```

El cuerpo de una clase sólo contiene los prototipos de las funciones miembro (las declaraciones). **La definición de la función se hace en los ficheros fuente (*.cpp).**

Para **definir la función miembro en el fichero fuente** se utiliza el nombre de la clase seguido por el ***operador de resolución de ámbito (::)***.

Ejemplo, en el fichero “**complejo.cpp**”:

```
void Complejo::Asignar(float x, float y)  
{  
    real = x; imaginario = y;           //acceso a los datos  
    ...  
}
```

Repaso - Ejemplo

Ejemplos de acceso a las variables (Ejercicio 1 de la práctica 6)

(Cuando las variables del objeto son “private”)

Forma 1

```
//file complex3.cpp
#include "complex3.h"

// member function SetData()
void complex::SetData(void)
{
    cout << "Real part: ";
    cin >> real;
    cout << "Imaginary part: ";
    cin >> imag;
```

```
//file main.cpp
#include "complex3.h"
void main(void)
{
    // complex numbers declarations
    complex c1,c2,c3;
    c1.SetData();
```

Forma 2

```
//file complex3.cpp
#include "complex3.h"

void complex::SetReal(double re)
{
    real = re;
}
void complex::SetImag(double im)
{
    imag = im;
}
```

```
//assign data to c4
double r,i;
cout << "Give r,i of c4:"; cin >> r >> i;
complex c4;
c4.SetReal(r); c4.SetImag(i);
```

Repaso - Ejemplo

Ejemplos de accesos a las variables (Ejercicio 1 de la práctica 6)

(Cuando las variables del objeto son “public”)

Forma 3

```
//fichero complejo.h

#include <iostream>
using namespace std;

class Complejo
{
public:
    float real, imaginario;
```

```
// fichero main.cpp
#include "Complejo.h"

void main()
{
    Complejo c1;
    c1.real = 4.5;
```

Repaso - Objeto

Un **Objeto** es un ejemplar concreto de una clase. Las **clases** son como **tipos de variables**, mientras que los **objetos** son como **variables concretas** de un tipo determinado.

¿Cómo se construye un objeto de una clase? Igual que cualquier otro objeto o variable de un tipo predefinido, es decir, anteponiendo el nombre de la clase al nombre del objeto:

Complejo miComplejo;

Un objeto **no se inicializa** como las variables:

Ejemplo: **Complejo miComplejo = 5 ;** **//Error**

Para solucionar este problema, C++ permite inicializar el objeto de una determinada clase en el momento de su declaración mediante un **constructor** definido por el usuario.

También: **comp1 = comp2;**

Para controlar el acceso a los miembros (datos y funciones) de una clase, C++ provee de 3 especificadores:

1. **public**: un miembro declarado público es **accesible en cualquier parte del programa** donde el **objeto** de la clase en cuestión es accesible.
2. **private**: un miembro declarado privado puede ser accedido solamente **por las funciones miembro de su propia clase o por funciones amigas (friend)** de su clase. En cambio **no puede ser accedido** por funciones globales o por las funciones propias de una clase derivada (**herencia**).
3. **protected**: un miembro declarado protegido se comporta exactamente igual que uno privado para las funciones globales, pero **actúa como un miembro público** para las funciones miembro de una clase derivada.

Repaso – Datos miembro

También **podemos declarar como datos miembro de una clase, *objetos de otra clase***, siendo necesario que ésta haya sido previamente definida.

```
class Datos{  
    Complejo c1;    //Declaración de un objeto  
                   //de la clase Complejo  
};
```

```
//fichero Cilindro.h  
#include "Circulo.h"  
  
class Cilindro {  
    Circulo base;  
    double altura;  
public:  
    void asignar(double r, double h)  
    {  
        base.asignar(r); altura = h;  
    }  
    double volumen() {  
        return base.area() * altura;  
    }  
};
```

```
//fichero Circulo.h  
class Circulo {  
    double radio;  
public:  
    void asignar(double r) { radio = r; }  
    double area() {return radio*radio*3.14159;}  
};
```

```
//fichero main.cpp  
#include "Cilindro.h"  
#include <iostream>  
using namespace std;  
  
void main (void) {  
    Cilindro c;  
    c.asignar(10,20);  
    cout << "Volumen del cilindro: ";  
    cout << c.volumen() << '\n';  
}
```

Ejemplo 7-1

1. Constructor

En C++, una forma de asegurar que los objetos siempre contengan valores válidos y que puedan ser inicializados al momento de la declaración es escribiendo un **constructor**.

Un **constructor** es una *función miembro especial* de una clase que es llamada de forma automática siempre que se declara un objeto de esa clase. Su función es crear e inicializar un objeto de su clase.

Dado que un constructor es una función miembro, **admite argumentos** al igual que éstas.

El **constructor** se puede distinguir claramente, con respecto a las demás funciones miembro de la clase, porque **tiene el mismo nombre que el de la clase**.

Un constructor **no retorna ningún valor ni se hereda**.

Si el usuario no ha creado uno, el compilador crea uno por omisión, sin argumentos.

Pueden existir **varios** constructores, siempre y cuando difieran en los argumentos.

2. Constructor

Ejemplo:

```
#include <iostream>
using namespace std;
#include "complejo.h"
void main()
{
    Complejo c1;      //creación de un objeto
    c1.Asignar(4.5,5.5); //Correcto
    c1.Print();

    //objeto c2
    Complejo c2;      //usa el const. defecto
    c2.Print();

    //objeto c3
    Complejo c3(1.5,-1.5); //usa el const. 2
    c3.Print();
}
```

Ejemplo 7-2

```
//fichero complejo.h
#include <iostream>
using namespace std;
class Complejo {
private:
    float real, imaginario;
public:
    void Asignar(float x, float y);
    void Print(void){
        cout <<real<<" + "<<imaginario<<"i\n";
    }
public:
    Complejo(){                          //constructor 1
        real=imaginario=0.0;}

    Complejo(float x, float y); // constructor 2
};
```

```
// fichero complejo.cpp
#include "complejo.h"
Complejo::Complejo(float x, float y){
    real = x;
    imaginario = y;
}
void Complejo::Asignar(float x, float y)
{
    real = x; imaginario = y;
}
```

3. Destructor

De la misma forma que existe una función para **construir** cada uno de los objetos que declaramos, también existe **una función para destruir cada objeto construido, liberando así la memoria que ocupa**. Esta función recibe el nombre de **destructor**.

Un objeto es destruido automáticamente al salir del ámbito en el que ha sido definido.

El **destructor** es una función miembro especial de una clase que se utiliza para eliminar un objeto de esa clase.

El **destructor** se distingue fácilmente del resto de funciones porque tiene **el mismo nombre que la clase precedido del operador tilde “~”**.

El **destructor** no es heredado, no tiene argumentos ni puede retornar ningún valor.

No hace falta llamar al destructor, ya que éste es invocado de forma automática cuando se destruye el objeto por los medios mencionados anteriormente.

Si hiciera falta, el destructor también puede ser invocado de forma explícita:

Ejemplo: `Objeto.nombre clase::~~nombre clase;` //llamada al destructor

```
//complejo.h
class Complejo {
public:
    float real, imaginario;
public:
    Complejo();
    //constructor
    ~Complejo();
    //destructor
};
```

```
//complejo.cpp
Complejo::Complejo(){
    real = imaginario = 0;
}
Complejo::~~Complejo(){
    cout << "Se ha llamado al
destruc\n";
}
```

4. Puntero implícito *this*

C++ proporciona este puntero para que una función miembro conozca la identidad del objeto particular que la está llamando. De esa forma, puede acceder a los datos de dicho objeto, ya que todos los objetos comparten las funciones miembro, más no los datos, los cuales cada uno tiene su propia copia.

```
#include "complejo.h"
void main(){
    Complejo c1(4.5,-1.5);
    Complejo c2;
    c2.Asignar(1.5,1.0);
    Complejo Sum;
    Sum = c1.Suma(c2);
    c1.Print();cout<<" + ";
    c2.Print();
    cout << " = ";
    Sum.Print(); cout << endl;
    //Resta
    Complejo Resta;
    Resta.Resta(c1,c2);
    c1.Print();cout<<" - ";
    c2.Print();
    cout << " = ";
    Resta.Print(); cout << endl;
}
```

```
//fichero complejo.h
#include <iostream>
using namespace std;
class Complejo{
private:
    float real,imag;
public:
    //constructores
    Complejo() { real = imag = 0.0;}
    Complejo(float x, float y);
    ~Complejo(); //destructor
public:
    void Asignar(float x, float y);
    void Print(void){
        cout
        << "[" << real << ", " << imag << "i]";
    }
    Complejo Suma(Complejo c);
    void Resta(Complejo,Complejo);
};
```

```
// fichero complejo.cpp
#include "complejo.h"
void Complejo::Asignar(float x, float y){
    real = x; imag = y;
}
Complejo::Complejo(float x, float y){
    real = x; imag = y;
}
Complejo::~Complejo(){
    cout << "Se ha llamado al destructor\n";
}
Complejo Complejo::Suma(Complejo c){
    Complejo tmp; //se crean 3 Compl. temp.
    tmp.real = real + c.real;
    tmp.imag = imag + c.imag;
    return tmp;
}
void Complejo::Resta(Complejo c1,Complejo c2){
    real = c1.real - c2.real;
    imag = c1.imag - c2.imag;
}
```

Ejemplo 7-3

5. Miembros *static* de una clase

Cada objeto tiene una copia de los datos miembro de la clase. **¿Qué pasa cuando necesito que todos los objetos tengan un dato miembro en común?**

Un *dato miembro* de una clase declarado como *static* implica que existirá una copia de ese dato miembro para todos los objetos.

Es un dato asociado con la clase y no con el objeto.

Un dato miembro *static*:

- Puede ser declarado *protected*, *private* o *public*.
- **Tiene que ser inicializado a nivel global** (ámbito de fichero, no de clase).
- Cuando sea **accedido a través de una función miembro**, dicha función **también tiene que ser declarada *static***.
- Existe aunque no haya ningún objeto de la clase declarado.

Ejemplo 7-4

```
#include <iostream>
using namespace std;
class CPersona
{
private:
    static int num;
    char *nombre;
    int edad;
public:
    CPersona(char *name="No Dato",int ed=0){
        nombre = name; edad = ed;
        num++;
    }
    ~CPersona(){ num--; }
    static int Num(){ return num;}
    char *GetName(){ return nombre; }
};
```

```
#include <iostream>
using namespace std;
#include "CPersona.h"
int CPersona::num = 0;    //inicializac. global

void main()
{
    CPersona p1("Carla",5);
    cout<<"Num.Objetos:"<<CPersona::Num()<<endl;
    CPersona p2("Juan",31);
    cout<<"Num.Objetos:"<<CPersona::Num()<<endl;
    CPersona *p3 = new CPersona("Ana",25);
    cout<<"Num.Objetos:"<<CPersona::Num()<<endl;
    delete p3;           //eliminar 1 objeto
    cout<<"Num.Objetos:"<<p1.Num()<<endl;
    cout<<"Nombre:"<<p1.GetName()<<endl;
}
```

6. Array's de objetos

Al igual que con los tipos estándar, también es posible crear **array's** de objetos:

- Forma **estática**, utilizando los corchetes: **Nombre_clase Nombre_array[num_objetos];**
- Forma **dinámica**, usando **new**: **Nombre_clase *Nom_Puntero = new Nombre_clase[num];**

Recordar de llamar a delete cuando ya no necesitemos el array.

Para crear un puntero a objeto se utiliza **new**. Para acceder a los miembros se usa el operador (->).

```
class CAumno
{
private:
    char *nombre;
    int edad;
    long telefono;
public:
    CAumno(){}; //const. por defecto
    CAumno(char *buf, int ed, int tel);
    ~CAumno();
    void Print();
    void Asignar(char *buf, int ed, int tel);
};
CAumno::CAumno(char *buf, int ed, int tel)
{
    nombre = new char[strlen(buf)+1];
    strcpy(nombre, buf);
    edad = ed;
    telefono = tel;
}
CAumno::~CAumno()
{
    if (nombre!=NULL) delete nombre;
}
```

```
void CAumno::Asignar(char *buf, int ed, int tel){
    nombre = new char[strlen(buf)+1];
    strcpy(nombre, buf);
    edad = ed;
    telefono = tel;
}
void CAumno::Print(){
    cout << "\nNombre: " << nombre;
    cout << "\nEdad: " << edad;
    cout << "\nTelefono: " << telefono;
}
void main(){
    CAumno al[3]; //array de 3 objetos
    al[0].Asignar("Juan", 22, 943215677);
    al[1].Asignar("Ana", 21, 943215271);
    for (int i=0; i<2; i++) al[i].Print();
    int num=2; // dinamicamente
    CAumno *alumnos = new CAumno[num];
    alumnos[0].Asignar("Carla", 5, 943567221);
    alumnos[1].Asignar("Mikel", 15, 943567222);
    for (i=0; i<num; i++) alumnos[i].Print();
    delete [] alumnos;
    CAumno *pAlum = new
        CAumno("Joseba", 21, 943765432);
    pAlum->Print();
    delete pAlum;
}
```

7. Funciones “amigas”

Los miembros de una clase declarados como privados solamente pueden ser accedidos mediante las funciones miembro de la clase, garantizando así la integridad de los datos.

Una función declarada **friend** (amiga) de una clase puede acceder a los miembros privados y protegidos de la clase C++.

Para declarar una función amiga, basta con escribir la palabra clave **friend** delante del nombre de la función. En la definición de ésta no hace falta anteponer el nombre de la clase. Véase el siguiente ejemplo:

```
class Complejo
{
private:
    float real;
    float imag;

public:
    Complejo(){};
    Complejo(float x, float y){
        real=x; imag=y;
    }
    void Print();
    friend void LeerDatos(double &r, double &i, Complejo c);
};
```

```
void LeerDatos(double &r, double &i, Complejo c)
{
    r=c.real; i=c.imag;
}
void Complejo::Print()
{
    cout << "Real: " << real ;
    cout << " Imag: " << imag <<endl;
}
void main()
{
    Complejo c1(1.5,4.5);
    double r,i;
    LeerDatos(r,i,c1); //accede a datos de c1
    cout << "R:"<<r<<" Imag:"<<i;
}
```